Qiskit Certification — Pauli Operators, Circuits & Gate Library

Answer Key with Full Reasoning (30 Questions)

Study Group Prep Deck

August 29, 2026

Answer Key — Overview

Full solutions with reasoning follow, one question per slide.

Companion to the 30-question "Pauli Operators, Circuits & Gate Library" practice PDF.

Answer Key — Question 1

Correct Answer: D

Y

Reasoning

The notebook's mapping table gives: $(z, x) = (0, 0) \to I$, $(0, 1) \to X$, $(1, 0) \to Z$, $(1, 1) \to Y$. This matches the identity $ZX = -iY$, so $(-i)^1 ZX = (-i)(-iY)$ reduces (up to the tracked phase q) to the Y operator.

Answer Key — Question 2

Correct Answer: C

Qubit 2 (the left-most position, most significant)

Reasoning

The notebook states explicitly: "Qubit-0 corresponds to the right-most Pauli character... `'XYZ'` represents $X \otimes Y \otimes Z$ with 'Z' on qubit-0, 'Y' on qubit-1, and 'X' on qubit-2." So the left-most character always acts on the highest-indexed qubit.

Answer Key — Question 3

Correct Answer: C

-1

Reasoning

Qiskit encodes the phase of a Pauli as an integer $q \in \mathbb{Z}_4$ where $0 \to +1$, $1 \to -i$, $2 \to -1$, $3 \to +i$. So $q = 2$ corresponds to an overall phase of -1.

Answer Key — Question 4

Correct Answer: C

Only $\{1, -1, 1j, -1j\}$

Reasoning

The notebook states: "Pauli can only be multiplied by 1, -1, 1j, -1j" — these are exactly the four phase values representable by the 2-bit phase index $q \in \{0, 1, 2, 3\}$; any other scalar (e.g. 0.5 or $1/\sqrt{2}$) is not a valid Pauli phase and would raise an error.

Answer Key — Question 5

Correct Answer: B

The Z/X bit pattern stays the same, but the phase is complex-conjugated (so $+i \to -i$)

Reasoning

The notebook notes: "For pauli op, adjoint is the same but phase may change." Since the underlying $Z^z X^x$ part is Hermitian up to phase, taking the conjugate transpose only conjugates the phase factor $(-i)^{q+z \cdot x}$, flipping $+i \leftrightarrow -i$ while leaving $Z^z X^x$ unchanged.

Answer Key — Question 6

Correct Answer: B

Whether the underlying X and Z Pauli strings anticommute, independent of the extra phase factor (and the number of qubits must match)

Reasoning

Anticommutation ($AB = -BA$) for Pauli strings is a property of the underlying $Z^z X^x$ structure (here X and Z anticommute), not of the scalar phase attached to one operand; the notebook also notes the two operators must act on the same number of qubits for the check to be meaningful.

Answer Key — Question 7

Correct Answer: B

Heisenberg picture: $P' = C^\dagger P C$

Reasoning

The notebook is explicit: `evolve` "Performs Heisenberg (default) or frame='h' ... Performs Schrödinger of the Pauli by a Clifford or frame='s'." The Heisenberg picture evolves as $P' = C^\dagger P C$, and this is what runs when `frame` is omitted.

Answer Key — Question 8

Correct Answer: B

`CircuitInstruction` objects, each containing the operation and the specific qubits/clbits it acts on

Reasoning

Per the notebook: "The data attribute provides direct access to the list of instructions that make up the circuit. Each item in the list is a `CircuitInstruction` object containing the operation and the qubits/clbits it applies to."

Answer Key — Question 9

Correct Answer: B

`append` always modifies `qc` in place and returns an `InstructionSet`; `compose` (without `inplace=True`) returns a *new* circuit and leaves the original `qc` unmodified

Reasoning

The notebook demonstrates this directly: `qc.append(XGate(), [0])` mutates `qc` and returns `InstructionSet`, while `qc1.compose(qc2, [1])` returns a new circuit object and `qc1` remains unchanged — "compose does not modify in-place by default."

Answer Key — Question 10

Correct Answer: B

`append` of a converted gate draws as a single opaque box (unless later decomposed); `compose` flattens the sub-circuit's gates inline into the drawing

Reasoning

The notebook states this directly: "`compose()` flattens the second circuit into the first so its gates appear inline. `append()` adds the other circuit as a single opaque instruction (a boxed gate)" — visually one is expanded, the other is a single labeled box.

Answer Key — Question 11

Correct Answer: B

It composes the two systems into a larger Hilbert space — equivalent to forming the tensor product of the two circuits' state spaces

Reasoning

Per the notebook: "`tensor` composes systems into a larger Hilbert space — i.e. it is equivalent to creating a larger system whose state space is the tensor product of the operand systems," unlike `compose`, which acts within a fixed qubit count.

Answer Key — Question 12

Correct Answer: B

The presence of extra idle qubits can let the transpiler use them as ancillas, generally reducing the number of basis gates needed to decompose the multi-controlled-X gate compared to the no-ancilla case

Reasoning

The notebook explicitly compares `small_circuit` (no idle qubits, 5 total) against `large_circuit` (10 qubits, 5 idle) both containing the same `MCXGate(4)`, then prints `count_ops()` for each after transpiling to `['u','cx']` — illustrating that available ancilla qubits change (typically reduce) the decomposition cost of multi-controlled gates.

Answer Key — Question 13

Correct Answer: B

No — a barrier is purely a directive that visually separates parts of the circuit and blocks transpiler optimizations across it; it has no effect on the circuit's unitary

Reasoning

The notebook describes `Barrier` simply as something that "visually separates different parts of the circuit." It is a non-operational directive (mainly used to prevent the transpiler from optimizing across it) and contributes no gate/unitary action of its own.

Answer Key — Question 14

Correct Answer: B

It measures the qubit in the computational basis, extracting the outcome and storing it onto a classical bit — this is a non-unitary, irreversible operation

Reasoning

Per the notebook, `measure` is "performed in computational basis, extracts quantum state and stores it onto a classical bit." Extraction of a definite classical outcome from a quantum superposition is inherently non-unitary and cannot be undone.

Answer Key — Question 15

Correct Answer: B

`reset` forces a qubit to $|0\rangle$ (non-unitary), while `initialize` can prepare an arbitrary (e.g. superposition) state and internally involves a reset followed by state preparation — also not a general unitary operation

Reasoning

The notebook notes `initialize` "is not a unitary operation in general and involves reset/prep," while `reset` simply forces a qubit to $|0\rangle$ mid-circuit as a non-unitary dynamic operation. `initialize` is the more general (and more expensive) of the two.

Answer Key — Question 16

Correct Answer: A

1 and 2

Reasoning

A `DiagonalGate` takes a list of 2^n diagonal entries for an n-qubit gate. A 2-element list implies $2^n = 2 \Rightarrow n = 1$ qubit; a 4-element list implies $2^n = 4 \Rightarrow n = 2$ qubits — exactly matching the notebook's two examples.

Answer Key — Question 17

Correct Answer: B

It marks qubit 8 as "used"/owned by that box block, even though no operation is applied to it

Reasoning

The notebook shows `with qc.box(): qc.cz(4,5); qc.cz(6,7); qc.noop(8)` specifically to illustrate marking qubit 8 as used by the box scope without performing any actual operation on it — useful for reserving/aligning qubit usage in control-flow blocks.

Answer Key — Question 18

Correct Answer: B

It exits the nearest enclosing loop early, conditioned on the classical value tested by `if_test`, enabling mid-circuit-measurement-driven dynamic control flow

Reasoning

`break_loop()` inside an `if_test` block, itself nested inside `for_loop`, implements conditional early termination of the loop based on a classical/measured condition — a core feature of Qiskit's dynamic-circuit control flow. (`continue_loop()`, by contrast, only skips to the next iteration rather than exiting the loop.)

Answer Key — Question 19

Correct Answer: B

`measure_active` measures only qubits that have been acted on (are "active"/non-idle) in the circuit, while `measure_all` adds measurements to *every* qubit, extending the classical register if necessary

Reasoning

The notebook's own comment on `measure_all` states it "Adds measurements to ALL qubits, extending the classical register if necessary," implicitly contrasting with `measure_active`, used earlier in the notebook, which targets only the qubits actually used/active in the circuit.

Answer Key — Question 20

Correct Answer: B

It strips trailing `measure` instructions (and associated unused classical resources) from the end of the circuit

Reasoning

The notebook applies `qc3.remove_final_measurements()` directly after `qc3.measure_active()` and shows the measurements disappearing from the drawn circuit — this method removes terminal measurement instructions (and can clean up now-unused classical registers).

Answer Key — Question 21

Correct Answer: B

15, since `width()` returns the total of qubits *plus* classical bits

Reasoning

`QuantumCircuit.width()` returns the number of qubits *plus* the number of classical bits. The notebook's assertion `qc.width()==12` holds only because that particular circuit has zero classical bits; adding 3 clbits would raise `width()` to $12 + 3 = 15$, a common exam trap for conflating `width()` with `num_qubits`.

Answer Key — Question 22

Correct Answer: B

The circuit's gates must all be within the basis gates/timing information described by the supplied `target`, which supplies the per-gate durations used in the estimate

Reasoning

The notebook passes `backend.target` (timing information) from a `GenericBackendV2` into `estimate_duration`; the method looks up each instruction's duration from that target's gate-timing data to estimate the total circuit duration.

Answer Key — Question 23

Correct Answer: B

Both remain at their default value of zero, because timing methods only return non-zero values after the circuit has been transpiled with an explicit `scheduling_method` such as `'alap'` or `'asap'`

Reasoning

The notebook is explicit: "these methods will only return non-zero values if the circuit has been transpiled with a `scheduling_method`... The `qc_scheduled` object... was created without a scheduling pass, so its timing information remains at the default of zero." This is a classic tricky exam distinction versus the separately-scheduled `qc_demo_scheduled` object.

Answer Key — Question 24

Correct Answer: B

The original qubit becomes the *target* and is moved to wire 1; the new control qubit occupies wire 0

Reasoning

The notebook is explicit about the reindexing: "The original qubit (0) becomes the target (now on wire 1). The new control is on wire 0." This wire-shuffling detail is easy to get backwards and is a common trick point.

Answer Key — Question 25

Correct Answer: B

`inverse()` reverses the gate order *and* replaces each gate with its own inverse (giving `Sdg` then `H`); `reverse_ops()` only reverses the instruction order without inverting any gate (giving `S` then `H`)

Reasoning

The notebook is explicit: `.inverse()` "returns a new circuit with all of the gates applied in the reverse order, and with each gate replaced by its own inverse," whereas `.reverse_ops()` "only reverses the order; it does not take the inverse of each gate." Confusing these two is one of the most common certification-exam traps on this topic.

Answer Key — Question 26

Correct Answer: B

An `S` gate (phase gate with $\theta = \pi/2$), because $T = \mathrm{diag}(1, e^{i\pi/4})$ so $T^2 = \mathrm{diag}(1, e^{i\pi/2}) = S$

Reasoning

The notebook demonstrates exactly this: "The output diagram will show an S-gate (Phase Gate where theta=$\pi/2$), because $T^2 = S$." `.power(n)` raises the gate's unitary matrix to the n-th power, and squaring the T gate's phase of $\pi/4$ doubles it to $\pi/2$, i.e. the S gate.

Answer Key — Question 27

Correct Answer: B

`reverse_bits()` relabels/reindexes the qubits (e.g. converting little-endian display to big-endian), leaving gate order and target logic unchanged, while `reverse_ops()` reverses the sequence of instructions but keeps each gate on its original qubit

Reasoning

These are two orthogonal transformations: `reverse_bits()` is explicitly about "managing Qiskit's bit ordering convention... endianness," remapping which wire is drawn/indexed as qubit 0 without touching gate order, whereas `reverse_ops()` reorders the instruction sequence without touching qubit indices. Conflating the two is a common exam mistake.

Answer Key — Question 28

Correct Answer: B

In `TwoLocal`, the rotation layer consists of single-qubit gates applied to *all* qubits and the entanglement layer uses strictly 2-qubit gates; in `NLocal`, circuit blocks may have *arbitrary* size (up to the number of qubits), not restricted to 1- or 2-qubit blocks

Reasoning

The notebook states this directly: "In NLocal, the circuit blocks can have arbitrary sizes (smaller equal to the number of qubits in the circuit), while in TwoLocal, the rotation layers are single qubit gates applied on all qubits and the entanglement layer uses two-qubit gates." `TwoLocal` is thus a more constrained special case.

Answer Key — Question 29

Correct Answer: B

ZZFM adds explicit two-qubit entangling gates (the $R_{ZZ}(\theta)$ rotation) between qubits, while ZFM consists only of alternating single-qubit Hadamard and phase-gate layers with no entanglement

Reasoning

Per the notebook, the ZFM "consists of alternating layers of Hadamard gates and phase gates" (no entanglement, a product state), while the ZZFM "extends the ZFM with the inclusion of two-qubit entangling gates, specifically the ZZ-rotation gate $R_{ZZ}(\theta)$" — and is explicitly "conjectured to be generally expensive to compute on a classical computer, unlike the ZFM."

Answer Key — Question 30

Correct Answer: B

Two features per qubit: one via an R_Y rotation (angle θ) and a second via an R_Z rotation (angle ϕ) applied to the same qubit

Reasoning

The notebook defines DAE as "a combination of angle encoding and phase encoding... allows two feature values to be encoded in a single qubit: θ with a Y-axis rotation, and ϕ with a z-axis rotation," explicitly giving "a 2-times reduction in the number of qubits required" compared to plain angle encoding.